
Object-Oriented Programming

Lukas Holik

lukasholik@cs.aau.dk

(based on slides of Gabriela Montoya, thanks)

Course Organization

- 10 lectures and exercises (approx.).
 - Crazy first two weeks - Thursday 8:15, Monday 8:15, Thursday 8:15, Friday 12:30, then only Thursdays
- A mini-project to be through the sessions
 - You can submit your solution and register for a feedback session
- Interaction during the lectures via quizzes and discussions
- You are welcome to provide feedback about the activities
- Ask about the subject or propose what we should talk about any time
 - Or via email
 - Or [Teams](#)
 - Scheduling individual/group meeting is also possible (and subject to availability)

Literature

- Sharan, Kishori, and Adam L. Davis. Beginning Java 17 Fundamentals: Object-Oriented Programming in Java 17. Apress, 2021.
 - [Available via the AAU library](#)
 - [Supplementary material available](#)
- Sarcar, Vaskaran. Java Design Patterns: A Hands-On Experience with Real-World Examples. 3rd edition. Apress, 2022.
 - [Available via the AAU library](#)
 - [Supplementary material available](#)
- Other references might be recommended as complementary material of specific lectures

Assessment

- Probably a multiple-choice test, with some full-text questions, we will see.
- Designed to be easy having done course activities and hard otherwise.
- It will have some specific overlap with course activities (e.g., similar tasks, similar domain,...)

Main Topics

- Classes
- Interfaces
- **Abstraction**
- **Encapsulation**
- **Inheritance**
- **Polymorphism**
- Design patterns

- The objective of this course is to learn about **these topics** and put that in practice using **Java**
- The main objective is **not** that you become experts in Java
- Writing programs in Java is not enough, the programs **must follow the principles of the object-oriented paradigm**

Java

- Java is The object oriented programming language (although it is strictly speaking not limited to object-oriented programming).
- Similar to other languages that you have used in previous semesters
- It is a strongly typed language (differently from JavaScript)
- Has garbage collector - a built-in mechanism for automatic memory allocation and deallocation (unlike C)
- Documentation
 - [API documentation](#) (you might want to look for existing classes)
 - [The Java Language Specification](#) (this reference can become more useful later)

High x Low level programming

- Instruct the computer to perform a task

Programs

Understandable by the
computer

```
0010010010 10010100000100110  
0001000100 01010010001001010
```

Machine language

Low level

```
li $t1, 15  
add $t0, $t1, 12
```

Assembly language

Easier to understand by
the programmer

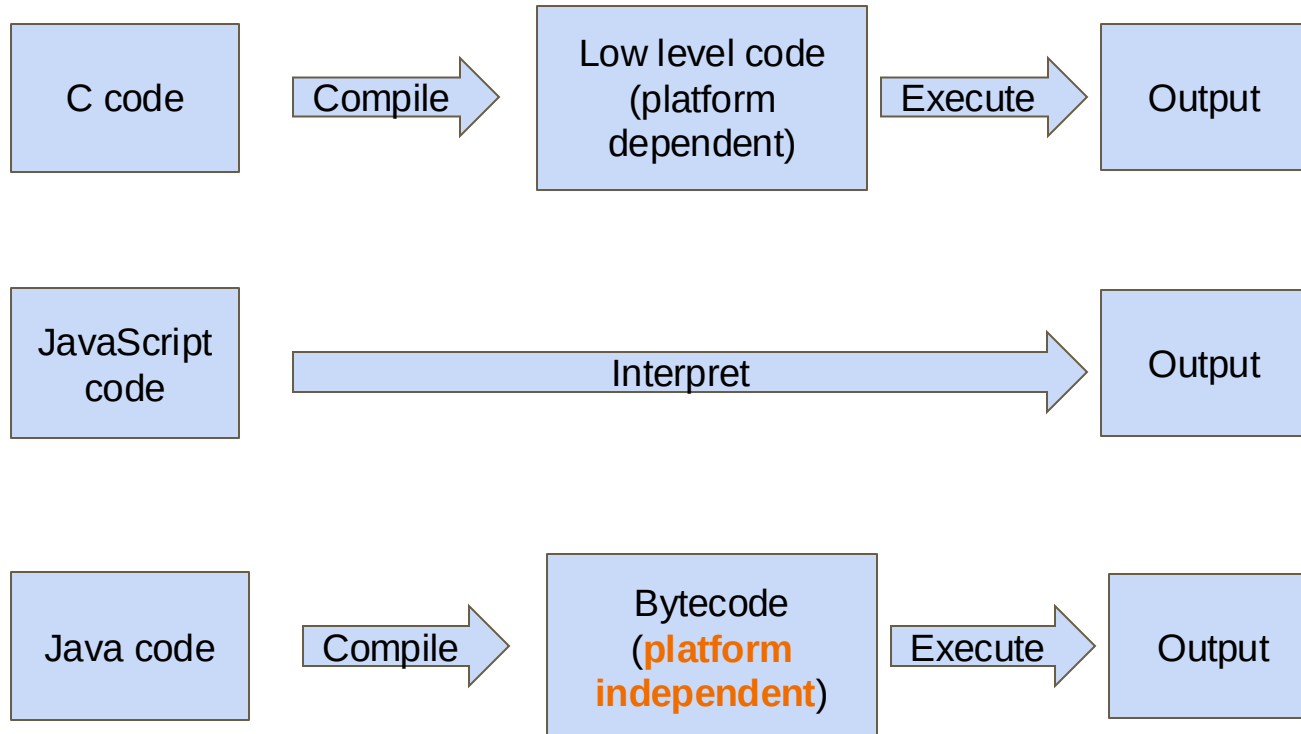
```
int x = 15 + 12;
```

High level

Task

Add two numbers: 15 and 12

Execution methods of high level programs



Quiz 1 - Background

Please provide your input using [this link](#)

Imperative Programming Paradigm

```
const courses = ['OOP', 'SU', 'DEB', 'ALG', 'IWP'];  
  
for (let i=0; i < courses.length; i++) {  
    if (courses[i]=='OOP') {  
        result = courses[i];  
    }  
}  
  
console.log(result);
```

Sequence of commands

Example in a programming language
used in the second semester

Programming paradigm = way to **conceptualize** about a problem
and its solution in the **underlying computational models**

Procedural Programming Paradigm

```
const courses = ['OOP', 'SU', 'DEB', 'ALG', 'IWP'];
```

```
function find(array, elem){  
  for (let i=0; i < array.length; i++) {  
    if (array[i]==elem) {  
      res = courses[i];  
    }  
  }  
  return res;  
}
```

```
result = find(courses, 'OOP');
```

```
console.log(result);
```

Example in a programming language
used in the second semester

A **procedure** combines multiple commands,
names and, parameterizes them,
is used as a new command.

Imperative and Procedural focus on **how** to find a solution.

Functional (Declarative) Programming Paradigm

```
const courses = ['OOP', 'SU', 'DEB', 'ALG', 'IWP'];  
  
const result = courses.find((c) => c == 'OOP');  
  
console.log(result);
```

Side effect free functions

Immutable values

Example in a programming language
used in the second semester

Focus on **what** is satisfied by a solution.

An imperative program in Java

```
const courses = ['OOP', 'SU', 'DEB', 'ALG', 'IWP'];  
  
for (let i=0; i < courses.length; i++) {  
  if (courses[i]=='OOP') {  
    result = courses[i];  
  }  
}  
  
console.log(result);
```

```
String[ ] courses = {"OOP", "SU", "DEB", "ALG", "IWP"};  
String result = null;
```

```
for (int i=0; i < courses.length; i++) {  
  if (courses[i]=="OOP") {  
    result = courses[i];  
  }  
}
```

```
System.out.println(result);
```

Use of types and array literals as in C

Statements similar to C and JavaScript

Available libraries to take care of many standard tasks

A procedural program in Java

```
String find(String[] array, String elem) {
```

```
    String res = null;
```

```
    for (int i=0; i < array.length; i++) {  
        if (array[i]==elem) {  
            res = array[i];  
        }  
    }
```

```
    return res;  
}
```

```
String[] courses = {"OOP", "SU", "DEB", "ALG", "IWP"};  
String result = find(courses, "OOP");
```

```
System.out.println(result);
```

```
const courses = ['OOP', 'SU', 'DEB', 'ALG', 'IWP'];
```

```
function find(array, elem){  
    for (let i=0; i < array.length; i++) {  
        if (array[i]==elem) {  
            res = courses[i];  
        }  
    }  
    return res;  
}
```

```
result = find(courses, 'OOP');
```

```
console.log(result);
```

A functional program in Java

```
const courses = ['OOP', 'SU', 'DEB', 'ALG', 'IWP'];  
  
const result = courses.find((c) => c == 'OOP');  
  
console.log(result);
```

```
String[] courses = {"OOP", "SU", "DEB", "ALG", "IWP", "SLIAL", "DTG", "IMPR", "PBL"};  
String result = Arrays.stream(courses).filter((c)-> c =="OOP").findAny().get();  
  
System.out.println(result);
```

An object-oriented program in Java

```
class Catalog {  
    private String[] elems;  
    private int size;  
  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        this.size = 0;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public String find(String e) {  
        String res = null;  
        for (int i=0; i < this.elems.length; i++) {  
            String currentElem = this.elems[i];  
            if (currentElem==e) {  
                res = currentElem;  
            }  
        }  
        return res;  
    }  
}
```

```
Catalog courses = new Catalog(45);  
courses.addElem("OOP");  
courses.addElem("SU");  
...  
courses.addElem("PBL");  
  
String result = courses.find("OOP");
```

Interacting **objects**

Objects **encapsulate** data and algorithms (**state** and **behavior**).

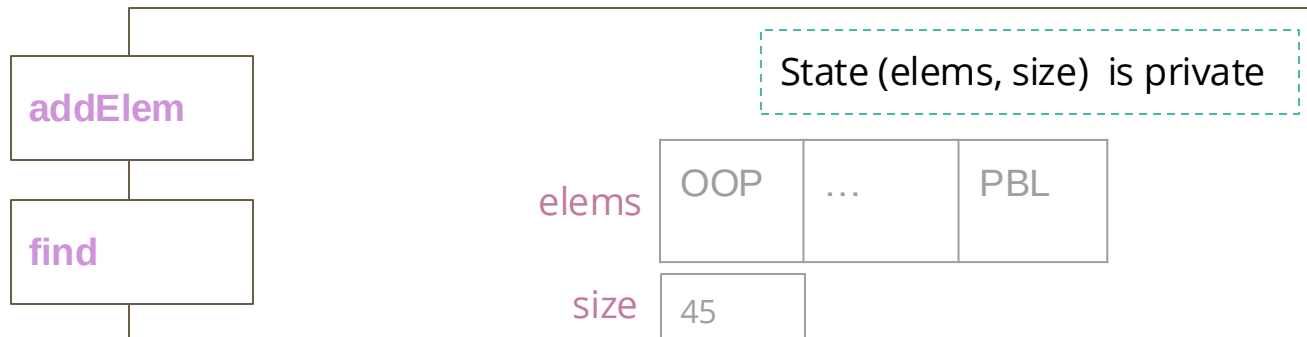
Constructors are used to create objects.

Interaction between objects is done by invoking their **methods**.

Quiz 2 - Paradigms

Please provide your input using [this link](#)

State and behavior of an object



elems and **size** are **instance variables**, the **object state**. Every instance of this class has its own state.

addElem and **find** are **instance methods**, the **object behavior**. Used to interact with this object.

Instance methods can access instance variables.

Quiz 3 - State and behavior

Please provide your input using [this link](#)

Principles of the object-oriented paradigm

- Abstraction
 - Focus on what is important, ignore details
- Encapsulation
 - Combine data and operations in an entity
- Inheritance
 - Derive new types
- Polymorphism
 - Behavior depends on the context

```
class Catalog {  
    private String[] elems;  
    private int size;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        this.size = 0;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public String find(String e) {  
        String res = null;  
        for (int i=0; i < this.elems.length; i++) {  
            String currentElem = this.elems[i];  
            if (currentElem==e) {  
                res = currentElem;  
            }  
        }  
        return res;  
    }  
}
```

Abstraction

- Large program is **decomposed** into subprograms.
- **Subprograms** are **simple, small, understandable, named**, are **independent** up to small and well-defined **interfaces** to interact with the world, can have specifications, are **reusable**, ...
- Allows to **focus on the relevant** parts, ignore irrelevant context.

Abstraction by parametrization

Sum numbers between 10 and 20.

```
int sum = 0;
int counter = 10;

while (counter <= 20) {
    sum = sum + counter;
    counter = counter + 1;
}

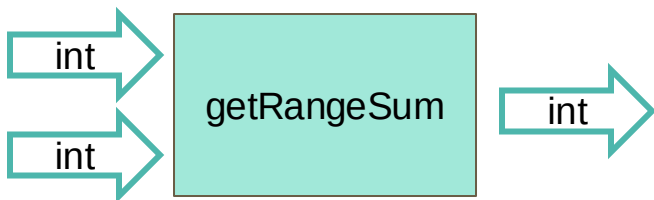
System.out.println(sum);
```

```
int getRangeSum(int lowerLimit, int upperLimit) {

    int sum = 0;
    int counter = lowerLimit;
    while (counter <= upperLimit) {
        sum = sum + counter;
        counter = counter + 1;
    }
    return sum;
}

int s1 = getRangeSum(10, 20);
System.out.println(s1);
```

Abstraction by specification



```
int s1 = getRangeSum(10, 20);  
System.out.println(s1);
```

getRangeSum

```
public static int getRangeSum(int lowerLimit,  
                              int upperLimit)
```

Computes and returns the sum of all integers between two integers specified by `lowerLimit` and `upperLimit` parameters. The `lowerLimit` parameter must be less than or equal to the `upperLimit` parameter. If the sum of all integers between the `lowerLimit` and the `upperLimit` exceeds the range of the `int` data type then result is not defined.

Parameters:

`lowerLimit` - The lower limit of the integer range

`upperLimit` - The upper limit of the integer range

Returns:

The sum of all integers between `lowerLimit` (inclusive) and `upperLimit` (inclusive)

Data abstraction

Catalog is a new data type (abstract data type, ADT)

```
class Catalog {  
    private String[] elems;  
    private int size;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        this.size = 0;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public String find(String e) {  
        String res = null;  
        for (int i=0; i < this.elems.length; i++) {  
            String currentElem = this.elems[i];  
            if (currentElem==e) {  
                res = currentElem;  
            }  
        }  
        return res;  
    }  
}
```

3 **public** member functions:
1 constructor and 2 methods

How the elements of the catalog are represented is **hidden** from the user.

`String[]` could be replaced by
`String`, `LinkedList`, `ArrayList`,...

and the code that uses Catalog continues to work without changes.

```
Catalog courses = new Catalog(45);  
courses.addElem("OOP");  
...
```

private hides elems (from Catalog's users)

Encapsulation

- Related data items **together in a container.**
- A class **combines data and operations** on them.
- Low level data and methods cannot be accessed from outside.

- Classes organized into **packages.**
- Packages organized into **modules.**

Encapsulation - Catalog of courses

```
class Catalog {  
    private String[] elems;  
    private int size;  
    public Catalog(int maxSize) {  
        this.elems = new String[maxSize];  
        this.size = 0;  
    }  
    public void addElem(String e) {  
        this.elems[size] = e;  
        this.size++;  
    }  
    public String find(String e) {  
        String res = null;  
        for (int i=0; i < this.elems.length; i++) {  
            String currentElem = this.elems[i];  
            if (currentElem==e) {  
                res = currentElem;  
            }  
        }  
        return res;  
    }  
}
```

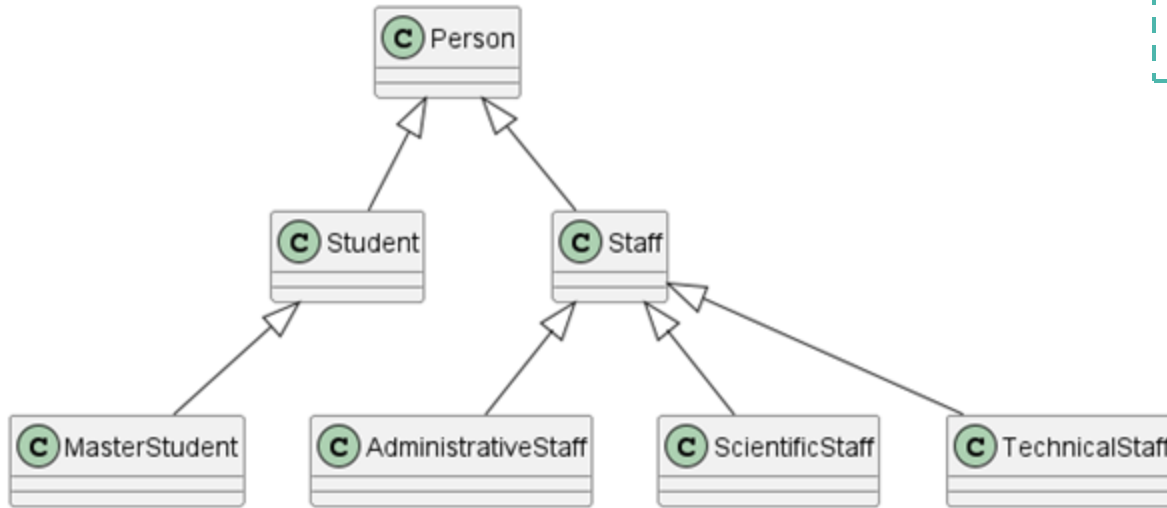
The data (**elems**, **size**) and operations (**constructor** and **methods**) are combined into one entity.

Inheritance

- Defining a new abstraction (type) by **extending** an existing one.
- A new **subtype** is created from an existing **supertype**.
- Subtypes **inherit** data and operations from their supertype.
- The supertype is a **generalization** of the subtype, the subtype is a **specialization** of the supertype.
- **Hierarchy** of types is created this way.

Inheritance - Person class and hierarchy of subtypes

Common state and behavior of all objects of these classes (in the supertype).



More specific / detailed state and behavior of objects of each subtype.

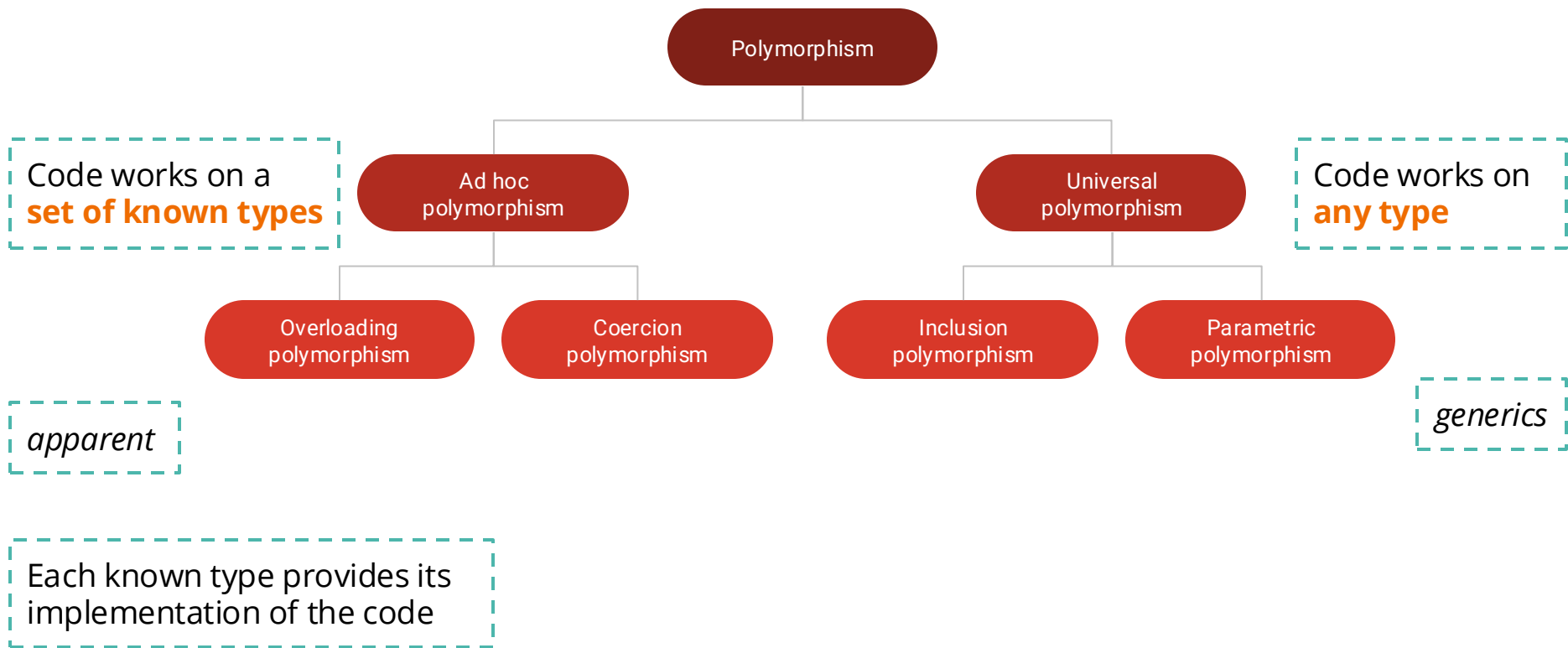
Quiz 4: Inheritance

Please provide your input using [this link](#)

Polymorphism

- Ability of an entity (e.g., class, method, object, parameter) to take on **different meanings in different contexts**.
- Allows for writing **reusable** and **maintainable** code.
- The code is said to be **generic**, i.e., it works for many types.

Polymorphism



Overloading polymorphism

```
class Catalog {  
    private Course[] elems;  
    private int size;  
    public Catalog(int maxSize) {  
        this.elems = new Course[maxSize];  
        this.size = 0;  
    }  
    public void addElem(String id) {  
        Course c = CourseFinder.find(id);  
        this.elems[size] = c;  
        this.size++;  
    }  
    public void addElem(Course c) {  
        this.elems[size] = c;  
        this.size++;  
    }  
}
```

```
swCatalog.addElem("OOP");
```

The types of the actual parameters of `addElem` determine which fragment of code will be executed

```
Course oop = ...  
swCatalog.addElem(oop);
```


Coercion polymorphism (vs. type casting)

```
double square(double num) {  
    return num * num;  
}
```

```
double d1 = 20.23;  
double result = square(d1);
```

```
int k = 2;  
double result = square(k);
```

```
int l = (int) square(2.3);
```

Coercion is **implicit conversion**.

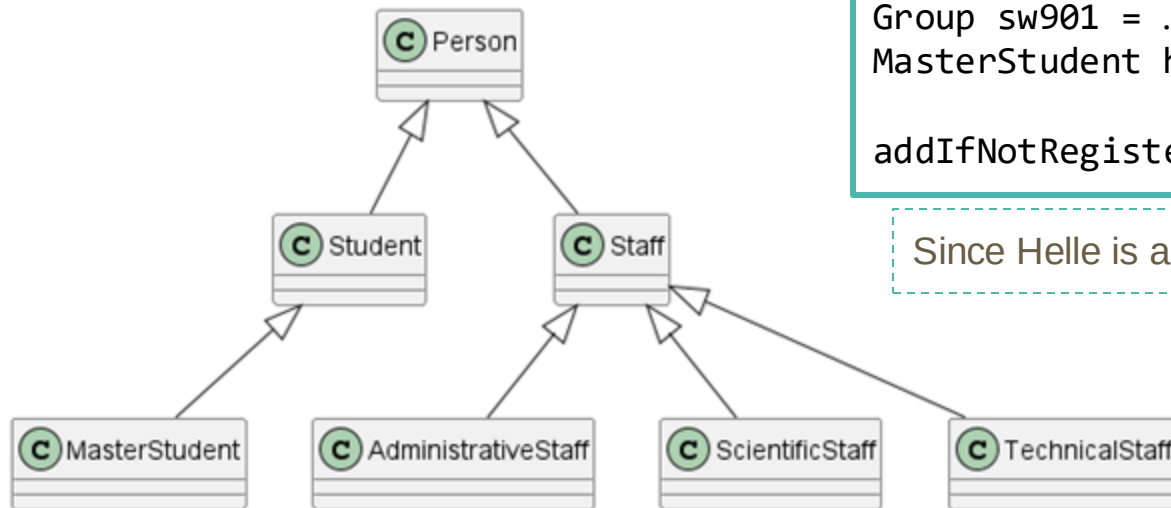
no conversion

widening conversion (int to double), **preserves info.**, **implicit** → it is **coercion polymorphism**

narrowing conversion (double to int), **looses info.**, must be **explicit** → not coercion, rather **type casting (not polymorphism)**

Implicit widening conversions available [here](#),
implicit narrowing conversions [here](#).

Inclusion polymorphism



The supertype **includes** all the values of its subtypes.

```
void addIfNotRegistered(Group g, Student s) {  
    if (!g.has(s)) {  
        g.add(s);  
    }  
}
```

```
Group sw901 = ...  
MasterStudent helle = ...
```

```
addIfNotRegistered(sw901, helle);
```

Since Helle is a MasterStudent, she is also a Student.

The code written for one type works for **all of its subtypes**.

Parametric polymorphism

Referred to as **generics**

```
class Catalog {  
    public ArrayList<Course> elems;  
    public Catalog() {  
        this.elems = new ArrayList<Course>();  
    }  
    public void addElem(Course c) {  
        this.elems.add(c);  
    }  
}
```

```
public class ArrayList<E>  
    extends AbstractList<E>  
    implements List<E>, RandomAccess, Cloneable, Serializable
```

E is a **type variable**

A **type variable** is used instead of any specific type

← → ↺ docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/ArrayList.html#add(E)

OVERVIEW MODULE PACKAGE **CLASS** USE TREE PREVIEW NEW DEPRECATED INDEX HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

add

```
public boolean add(E e)
```

Appends the specified element to the end of this list.

Specified by:

add in interface Collection<E>

Specified by:

add in interface List<E>

Overrides:

add in class AbstractList<E>

Parameters:

e - element to be appended to this list

Returns:

true (as specified by Collection.add(E))

Quiz 5: Polymorphism

Please provide your input using [this link](#)

Introduction to Object-Oriented Programming

- Information about the course
- First glance at Java
- Programming paradigms
- Object-oriented programming concepts
- Principles of the object-oriented paradigm

In Chapters 2 and 3 in [the book](#) you can find detailed descriptions that might help with these tasks, e.g., installation and using jshell, javac, java, or an IDE

Next Activities

Check [“Information about the exercises”](#) in Moodle (where to work today?)

1. Download and install Java from [here](#)
 - a. Notice the [installation instructions](#) available there
2. Try small fragments of code using jshell
 - a. Open a terminal and use the command **jshell**
 - b. Write or copy some fragments of a Java program (e.g., from previous slides or the book)
 - c. Modify existing fragments to obtain a different output
3. [Compile and execute](#) Java programs from a terminal
 - a. In [this folder](#) you can find a few programs to use
 - b. You might also choose some programs from the book's supplementary material
4. Download and install an IDE you like

IntelliJ-IDEA is nice, the book uses NetBeans, Visual Studio Code is popular, ...

Appendix

Programming Paradigms

```

class Catalog {
  #elems;
  #size;
  constructor() {
    this.#elems = [];
    this.#size = 0;
  }
  addElem(e) {
    this.#elems[this.#size] = e;
    this.#size = this.#size + 1;
  }
  find(e) {
    let res;
    for (let i=0; i < this.#size; i++) {
      if (this.#elems[i]==e) {
        res = this.#elems[i];
      }
    }
    return res;
  }
}

```

```

let courses = new Catalog();
courses.addElem('OOP');
courses.addElem('SU');
...
courses.addElem('PBL');

```

```
const result = courses.find('OOP');
```

Objects **encapsulate** data and algorithms
(state and behavior)

Classes are used to create objects
(blueprint)

Interaction between objects is done by
invoking their **methods**

Interacting **objects**

**Object
Oriented**